

Finding the voice of the user

Jeremy walked into the office of Ruth Gilbert, the director of the Drug Discovery Division at Contoso Pharmaceuticals. Ruth had asked the information technology team that supported Contoso's research organization to build a new application to help the research chemists accelerate their exploration for new drugs. Jeremy was assigned as the business analyst for the project. After introducing himself and discussing the project in broad terms, Jeremy said to Ruth, "I'd like to talk with some of your chemists to understand their requirements for the system. Who might be some good people to start with?"

Ruth replied, "I did that same job for five years before I became the division director three years ago. You don't really need to talk to any of my people; I can tell you everything you need to know about this project."

Jeremy was concerned. Scientific knowledge and technologies change quickly, so he wasn't sure if Ruth could adequately represent the current and future needs for users of this complex application. Perhaps there were some internal politics going on that weren't apparent and there was a good reason for Ruth to create a buffer between Jeremy and the actual users. After some discussion, though, it became clear that Ruth didn't want any of her people involved directly with the project.

"Okay," Jeremy agreed reluctantly. "Maybe I can start by doing some document analysis and bring questions I have to you. Can we set up a series of interviews for the next couple of weeks so I can understand the kinds of things you expect your scientists to be able to do with this new system?"

"Sorry, I'm swamped right now," Ruth told him. "I can give you a couple of hours in about three weeks to clarify things you're unsure about. Just go ahead and start writing the requirements. When we meet, then you can ask me any questions you still have. I hope that will let you get the ball rolling on this project."

If you share our conviction that customer involvement is a critical factor in delivering excellent software, you will ensure that the business analyst (BA) and project manager for your project will work hard to engage appropriate customer representatives from the outset. Success in software requirements, and hence in software development, depends on getting the voice of the user close to the ear of the developer. To find the voice of the user, take the following steps:

- Identify the different classes of users for your product.
- Select and work with individuals who represent each user class and other stakeholder groups.
- Agree on who the requirements decision makers are for your project.

Customer involvement is the best way to avoid the expectation gap described in Chapter 2, “Requirements from the customer’s perspective,” a mismatch between the product that customers expect to receive and what developers build. It’s not enough simply to ask a few customers or their manager what they want once or twice and then start coding. If developers build exactly what customers initially request, they’ll probably have to build it again because customers often don’t know what they really need. In addition, the BAs might not be talking to the right people or asking the right questions.

The features that users present as their “wants” don’t necessarily equate to the functionality they need to perform their tasks with the new product. To gain a more accurate view of user needs, the business analyst must collect a wide range of user input, analyze and clarify it, and specify just what needs to be built to let users do their jobs. The BA has the lead responsibility for recording the new system’s necessary capabilities and properties and for communicating that information to other stakeholders. This is an iterative process that takes time. If you don’t invest the time to achieve this shared understanding—this common vision of the intended product—the certain outcomes are rework, missed deadlines, cost overruns, and customer dissatisfaction.

User classes

People often talk about “the user” for a software system as though all users belong to a monolithic group with similar characteristics and needs. In reality, most products of any size appeal to a diversity of users with different expectations and goals. Rather than thinking of “the user” in singular, spend some time identifying the multiple user classes and their roles and privileges for your product.

Classifying users

Chapter 2 described many of the types of stakeholders that a project might have. As shown in Figure 6-1, a user class is a subset of the product’s users, which is a subset of the product’s customers, which is a subset of its stakeholders. An individual can belong to multiple user classes. For example, an application’s administrator might also interact with it as an ordinary user at times. A product’s users might differ—among other ways—in the following respects, and you can group users into a number of distinct *user classes* based on these sorts of differences:

- Their access privilege or security levels (such as ordinary user, guest user, administrator)
- The tasks they perform during their business operations
- The features they use
- The frequency with which they use the product
- Their application domain experience and computer systems expertise
- The platforms they will be using (desktop PCs, laptop PCs, tablets, smartphones, specialized devices)

- Their native language
- Whether they will interact with the system directly or indirectly

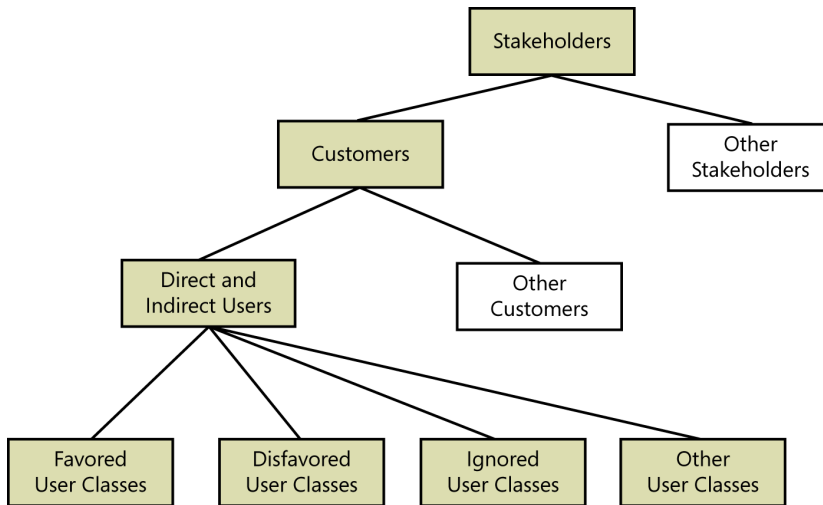


FIGURE 6-1 A hierarchy of stakeholders, customers, users, and user classes.



It's tempting to group users into classes based on their geographical location or the kind of company they work in. One company that creates software used in the banking industry initially considered distinguishing users based on whether they worked in a large commercial bank, a small commercial bank, a savings and loan institution, or a credit union. These distinctions really represent different market segments, though, not different user classes.

A better way to identify user classes is to think about the tasks that various users will perform with the system. All of those types of financial institutions will have tellers, employees who process loan applications, business bankers, and so forth. The individuals who perform such activities—whether they are job titles or simply roles—will have similar functional needs for the system across all of the financial institutions. Tellers all have to do more or less the same things, business bankers do more or less the same things, and so on. More logical user class names for a banking system therefore might include teller, loan officer, business banker, and branch manager. You might discover additional user classes by thinking of possible use cases, user stories, and process flows and who might perform them.

Certain user classes could be more important than others for a specific project. Favored user classes are those whose satisfaction is most closely aligned with achieving the project's business objectives. When resolving conflicts between requirements from different user classes or making priority decisions, favored user classes receive preferential treatment. This doesn't mean that the customers who are paying for the system (who might not be users at all) or those who have the most political clout should necessarily be favored. It's a matter of alignment with the business objectives.

Disfavored user classes are groups who aren't supposed to use the product for legal, security, or safety reasons (Gause and Lawrence 1999). You might build in features to deliberately make it hard for disfavored users to do things they aren't supposed to do. Examples include access security

mechanisms, user privilege levels, antimalware features (for non-human users), and usage logging. Locking a user's account after four unsuccessful login attempts protects against access by the disfavored user class of "user impersonators," albeit at the risk of inconveniencing forgetful legitimate users. If my bank doesn't recognize the computer I'm using, it sends me an email message with a one-time access code I have to enter before I can log on. This feature was implemented because of the disfavored user class of "people who might have stolen my banking information."

You might elect to ignore still other user classes. Yes, they will use the product, but you don't specifically build it to suit them. If there are any other groups of users that are neither favored, disfavored, nor ignored, they are of equal importance in defining the product's requirements.

Each user class will have its own set of requirements for the tasks that members of the class must perform. There could be some overlap between the needs of different user classes. Tellers, business bankers, and loan officers all might have to check a bank customer's account balance, for instance. Different user classes also could have different quality expectations, such as usability, that will drive user interface design choices. New or occasional users are concerned with how easy the system is to learn. Such users like menus, graphical user interfaces, uncluttered screen displays, wizards, and help screens. As users gain experience with the system, they become more interested in efficiency. They now value keyboard shortcuts, customization options, toolbars, and scripting facilities.

Trap Don't overlook indirect user classes. They won't use your application themselves, instead accessing its data or services through other applications or through reports. Your customer once removed is still your customer.

User classes need not be human beings. They could be software agents performing a service on behalf of a human user, such as bots. Software agents can scan networks for information about goods and services, assemble custom news feeds, process your incoming email, monitor physical systems and networks for problems or intrusions, or perform data mining. Internet agents that probe websites for vulnerabilities or to generate spam are a type of disfavored non-human user class. If you identify these sorts of disfavored user classes, you might specify certain requirements not to meet their needs but rather to thwart them. For instance, website tools such as CAPTCHA that validate whether a user is a human being attempt to block such disruptive access by "users" you want to keep out.

Remember, users are a subset of customers, which are a subset of stakeholders. You'll need to consider a much broader range of potential sources of requirements than just direct and indirect user classes. For instance, even though the development team members aren't end users of the system they're building, you need their input on internal quality attributes such as efficiency, modifiability, portability, and reusability, as described in Chapter 14, "Beyond functionality." One company found that every installation of their product was an expensive nightmare until they introduced an "installer" user class so they could focus on requirements such as the development of a customization architecture for their product. Look well beyond the obvious end users when you're trying to identify stakeholders whose requirements input is necessary.



Identifying your user classes

Identify and characterize the different user classes for your product early in the project so you can elicit requirements from representatives of each important class. A useful technique for this is a collaboration pattern developed by Ellen Gottesdiener called “expand then contract” (Gottesdiener 2002). Start by asking the project sponsor who he expects to use the system. Then brainstorm as many user classes as you can think of. Don’t get nervous if there are dozens at this stage; you’ll condense and categorize them later. It’s important not to overlook a user class, which can lead to problems later when someone complains that the delivered solution doesn’t meet her needs. Next, look for groups with similar needs that you can either combine or treat as a major user class with several subclasses. Try to pare the list down to about 15 or fewer distinct user classes.



One company that developed a specialized product for about 65 corporate customers initially regarded each company as a distinct user with unique needs. Grouping their customers into just six user classes greatly simplified their requirements challenges. Donald Gause and Gerald Weinberg (1989) offer much advice about casting a wide net to identify potential users, pruning the user list, and seeking specific users to participate in the project.

Various analysis models can help you identify user classes. The external entities shown outside your system on a context diagram (see Chapter 5, “Establishing the business requirements”) are candidates for user classes. A corporate organization chart can also help you discover potential users and other stakeholders (Beatty and Chen 2012). Figure 6-2 illustrates a portion of the organization chart for Contoso Pharmaceuticals. Nearly all of the potential users for the system are likely to be found somewhere in this chart. While performing stakeholder and user analysis, study the organization chart to look for:

- Departments that participate in the business process.
- Departments that are affected by the business process.
- Departments or role names in which either direct or indirect users might be found.
- User classes that span multiple departments.
- Departments that might have an interface to external stakeholders outside the company.

Organization chart analysis reduces the likelihood that you will overlook an important class of users within that organization. It shows you where to seek potential representatives for specific user classes, as well as helping determine who the key requirements decision makers might be. You might find multiple user classes with diverse needs within a single department. Conversely, recognizing the same user class in multiple departments can simplify requirements elicitation. Studying the organization chart helps you judge how many user representatives you’ll need to work with to feel confident that you thoroughly understand the broad user community’s needs. Also try to understand what type of information the users from each department might supply based on their role in the organization and their department’s perspective on the project.

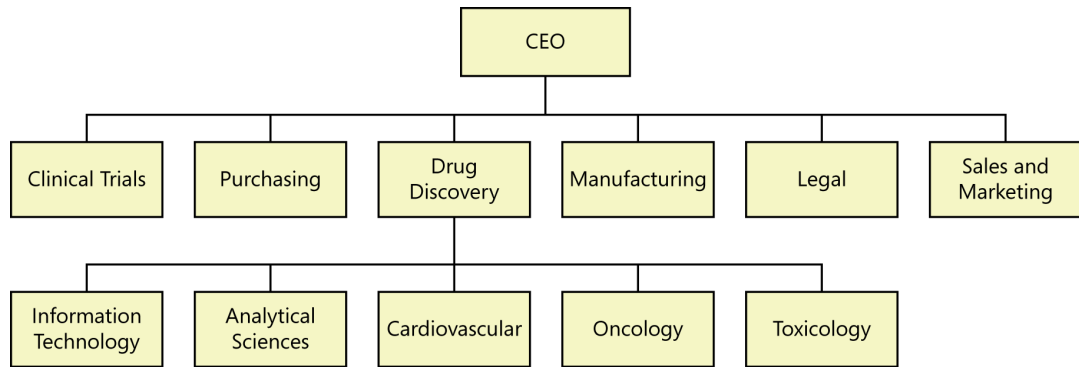


FIGURE 6-2 A portion of the organization chart for Contoso Pharmaceuticals.

Document the user classes and their characteristics, responsibilities, and physical locations in the software requirements specification (SRS) or in a requirements plan for your project. Check that information against any information you might already have about stakeholder profiles in the vision and scope document to avoid conflicts and duplication. Include all pertinent information you have about each user class, such as its relative or absolute size and which classes are favored. This will help the team prioritize change requests and conduct impact assessments later on. Estimates of the volume and type of system transactions help the testers develop a usage profile for the system so that they can plan their verification activities. The project manager and business analyst of the Chemical Tracking System discussed in earlier chapters identified the user classes and characteristics shown in Table 6-1.

TABLE 6-1 User classes for the Chemical Tracking System

Name	Number	Description
Chemists (favored)	Approximately 1,000 located in 6 buildings	Chemists will request chemicals from vendors and from the chemical stockroom. Each chemist will use the system several times per day, mainly for requesting chemicals and tracking chemical containers into and out of the laboratory. The chemists need to search vendor catalogs for specific chemical structures imported from the tools they use for drawing structures.
Buyers	5	Buyers in the purchasing department process chemical requests. They place and track orders with external vendors. They know little about chemistry and need simple query facilities to search vendor catalogs. Buyers will not use the system's container-tracking features. Each buyer will use the system an average of 25 times per day.
Chemical stockroom staff	6 technicians, 1 supervisor	The chemical stockroom staff manages an inventory of more than 500,000 chemical containers. They will supply containers from three stockrooms, request new chemicals from vendors, and track the movement of all containers into and out of the stockrooms. They are the only users of the inventory-reporting feature. Because of their high transaction volume, features that are used only by the chemical stockroom staff must be automated and efficient.
Health and Safety Department staff (favored)	1 manager	The Health and Safety Department staff will use the system only to generate predefined quarterly reports that comply with federal and state chemical usage and disposal reporting regulations. The Health and Safety Department manager will request changes in the reports periodically as government regulations change. These report changes are of the highest priority, and implementation will be time critical.

Consider building a catalog of user classes that recur across multiple applications. Defining user classes at the enterprise level lets you reuse those user class descriptions in future projects. The next system you build might serve the needs of some new user classes, but it probably will also be used by user classes from your earlier systems. If you do include the user-class descriptions in the project's SRS, you can incorporate entries from the reusable user-class catalog by reference and just write descriptions of any new groups that are specific to that application.

User personas

To help bring your user classes to life, consider creating a *persona* for each one, a description of a representative member of the user class (Cooper 2004; Leffingwell 2011). A persona is a description of a hypothetical, generic person who serves as a stand-in for a group of users having similar characteristics and needs. You can use personas to help you understand the requirements and to design the user experience to best meet the needs of specific user communities.

A persona can serve as a placeholder when the BA doesn't have an actual user representative at hand. Rather than having progress come to a halt, the BA can envision a persona performing a particular task or try to assess what the persona's preferences would be, thereby drafting a requirements starting point to be confirmed when an actual user is available. Persona details for a commercial customer include social and demographic characteristics and behaviors, preferences, annoyances, and similar information. Make sure the personas you create truly are representative of their user class, based on market, demographic, and ethnographic research.

Here's an example of a persona for one user class on the Chemical Tracking System:

Fred, 41, has been a chemist at Contoso Pharmaceuticals since he received his Ph.D. 14 years ago. He doesn't have much patience with computers. Fred usually works on two projects at a time in related chemical areas. His lab contains approximately 300 bottles of chemicals and gas cylinders. On an average day, he'll need four new chemicals from the stockroom. Two of these will be commercial chemicals in stock, one will need to be ordered, and one will come from the supply of proprietary Contoso chemical samples. On occasion, Fred will need a hazardous chemical that requires special training for safe handling. When he buys a chemical for the first time, Fred wants the material safety data sheet emailed to him automatically. Each year, Fred will synthesize about 20 new proprietary chemicals to go into the stockroom. Fred wants a report of his chemical usage for the previous month to be generated automatically and sent to him by email so that he can monitor his chemical exposure.

As the business analyst explores the chemists' requirements, he can think about Fred as the archetype of this user class and ask himself, "What would Fred need to do?" Working with a persona makes the requirements thought process more tangible than if you simply contemplate what a whole faceless group of people might want. Some people choose a random human face of the appropriate gender to make a persona seem even more real.



Dean Leffingwell (2011) suggests that you design the system to make it easy for the individual described in your persona to use the application. That is, you focus on meeting that one (imaginary) person's needs. Provided you've created a persona that accurately represents the user class, this should help you do a good job of satisfying the needs and expectations of the whole class. As one colleague related, "On a project for servicing coin-operated vending machines, I introduced Dolly the Serviceperson and Ralph the Warehouse Supervisor. We wrote scenarios for them and they became part of the project team—virtually."

Connecting with user representatives

Every kind of project—corporate information systems, commercial applications, embedded systems, websites, contracted software—needs suitable representatives to provide the voice of the user. These users should be involved throughout the development life cycle, not just in an isolated requirements phase at the beginning of the project. Each user class needs someone to speak for it.

It's easiest to gain access to actual users when you're developing applications for deployment within your own company. If you're developing commercial software, you might engage people from your beta-testing or early-release sites to provide requirements input much earlier in the development process. (See the "External product champions" section later in this chapter). Consider setting up focus groups of current users of your products or your competitors' products. Instead of just guessing at what your users might want, ask some of them.



One company asked a focus group to perform certain tasks with various digital cameras and computers. The results indicated that the company's camera software took too long to perform the most common operation because of a design decision that was made to accommodate less likely scenarios as well. The company changed their next camera to reduce customer complaints about speed.

Be sure that the focus group represents the kinds of users whose needs should drive your product development. Include both expert and less experienced customers. If your focus group represents only early adopters or blue-sky thinkers, you might end up with many sophisticated and technically difficult requirements that few customers find useful.

Figure 6-3 illustrates some typical communication pathways that connect the voice of the user to the ear of the developer. One study indicated that employing more kinds of communication links and more direct links between developers and users led to more successful projects (Keil and Carmel 1995). The most direct communication occurs when developers can talk to appropriate users themselves, which means that the developer is also performing the business analyst role. This can work on very small projects, provided the developer involved has the appropriate BA skills, but it doesn't scale up to large projects with thousands of potential users and dozens of developers.

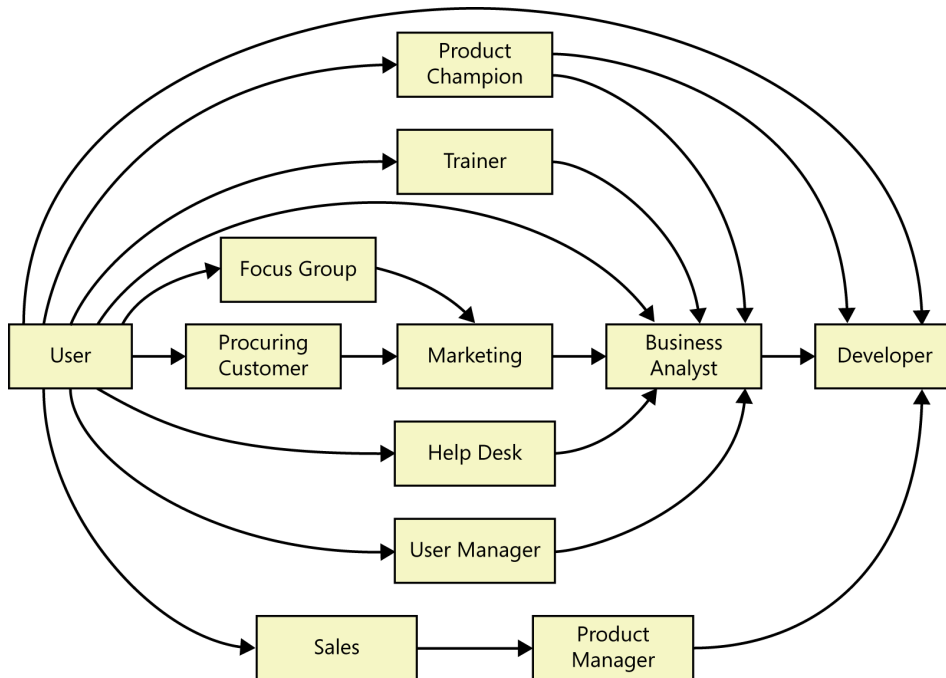


FIGURE 6-3 Some possible communication pathways between the user and the developer.

As in the children's game "Telephone," intervening layers between the user and the developer increase the chance of miscommunication and delay transmission. Some of these intervening layers add value, though, as when a skilled BA works with users or other participants to collect, evaluate, refine, and organize their input. Recognize the risks that you assume by using marketing staff, product managers, subject matter experts, or others as surrogates for the actual voice of the user. Despite the obstacles to—and the cost of—optimizing user representation, your product and your customers will suffer if you don't talk to the people who can provide the best information.

The product champion



Many years ago I worked in a small software development group that supported the scientific research activities at a major corporation. Each of our projects included a few key members of our user community to provide the requirements. We called these people *product champions* (Wiegiers 1996). The product champion approach provides an effective way to structure that all-important customer-development collaborative partnership discussed in Chapter 2.

Each product champion serves as the primary interface between members of a single user class and the project's business analyst. Ideally, the champions will be actual users, not surrogates such as funding sponsors, marketing staff, user managers, or software developers imagining themselves to be users. Product champions gather requirements from other members of the user classes they represent and reconcile inconsistencies. Requirements development is thus a shared responsibility of the BA and selected users, although the BA should actually write the requirements documents. It's hard enough to write good requirements if you do it for a living; it is not realistic to expect users who have never written requirements before to do a good job.

The best product champions have a clear vision of the new system. They're enthusiastic because they see how it will benefit them and their peers. Champions should be effective communicators who are respected by their colleagues. They need a thorough understanding of the application domain and the solution's operating environment. Great product champions are in demand for other assignments, so you'll have to build a persuasive case for why particular individuals are critical to project success. For example, product champions can lead adoption of the application by the user community, which might be a success metric that managers will appreciate. We have found that good product champions made a huge difference in our projects, so we offer them public reward and recognition for their contributions.



Our software development teams enjoyed an additional benefit from the product champion approach. On several projects, we had excellent champions who spoke out on our behalf with their colleagues when the customers wondered why the software wasn't done yet. "Don't worry about it," the champions told their peers and their managers. "I understand and agree with the software team's approach to software engineering. The time we're spending on requirements will help us get the system we really need and will save time in the long run." Such collaboration helps break down the tension that can arise between customers and development teams.

The product champion approach works best if each champion is fully empowered to make binding decisions on behalf of the user class he represents. If a champion's decisions are routinely overruled by others, his time and goodwill are being wasted. However, the champions must remember that they are not the sole customers. Problems arise when the individual filling this critical liaison role doesn't adequately communicate with his peers and presents only his own wishes and ideas.

External product champions

When developing commercial software, it can be difficult to find product champions from outside your company. Companies that develop commercial products sometimes rely on internal subject matter experts or outside consultants to serve as surrogates for actual users, who might be unknown or difficult to engage. If you have a close working relationship with some major corporate customers, they might welcome the opportunity to participate in requirements elicitation. You might give external product champions economic incentives for their participation. Consider offering them discounts on the product or paying for the time they spend working with you on requirements. You still face the challenge of how to avoid hearing only the champions' requirements and overlooking the needs of other stakeholders. If you have a diverse customer base, first identify core requirements that are common to all customers. Then define additional requirements that are specific to individual corporate customers, market segments, or user classes.



Another alternative is to hire a suitable product champion who has the right background. One company that developed a retail point-of-sale and back-office system for a particular industry hired three store managers to serve as full-time product champions. As another example, my longtime family doctor, Art, left his medical practice to become the voice-of-the-physician at a medical software company. Art's new employer believed that it was worth the expense to hire a doctor to help the company build software that other doctors would accept. A third company hired several former employees from one of their major customers. These people provided valuable domain expertise as well as insight into the politics of the customer organization. To illustrate an alternative engagement model, one company had several corporate customers that used their invoicing systems extensively. Rather than bringing in product champions from the customers, the developing company sent BAs to the customer sites. Customers willingly dedicated some of their staff time to helping the BAs get the right requirements for the new invoicing system.

Anytime the product champion is a former or simulated user, watch out for disconnects between the champion's perceptions and the current needs of real users. Some domains change rapidly, whereas others are more stable. Regardless, if people aren't operating in the role anymore, they simply might have forgotten the intricacies of the daily job. The essential question is whether the product champion, no matter what her background or current job, can accurately represent the needs of today's real users.

Product champion expectations

To help the product champions succeed, document what you expect your champions to do. These written expectations can help you build a case for specific individuals to fill this critical role. Table 6-2 identifies some activities that product champions might perform (Wiegers 1996). Not every champion will do all of these; use this table as a starting point to negotiate each champion's responsibilities.

TABLE 6-2 Possible product champion activities

Category	Activities
Planning	■ Refine the scope and limitations of the product. ■ Identify other systems with which to interact. ■ Evaluate the impact of the new system on business operations. ■ Define a transition path from current applications or manual operations. ■ Identify relevant standards and certification requirements.
Requirements	■ Collect input on requirements from other users. ■ Develop usage scenarios, use cases, and user stories. ■ Resolve conflicts between proposed requirements within the user class. ■ Define implementation priorities. ■ Provide input regarding performance and other quality requirements. ■ Evaluate prototypes. ■ Work with other decision makers to resolve conflicts among requirements from different stakeholders. ■ Provide specialized algorithms.

Category	Activities
Validation and verification	■ Review requirements specifications. ■ Define acceptance criteria. ■ Develop user acceptance tests from usage scenarios. ■ Provide test data sets from the business. ■ Perform beta testing or user acceptance testing.
User aids	■ Write portions of user documentation and help text. ■ Contribute to training materials or tutorials. ■ Demonstrate the system to peers.
Change management	■ Evaluate and prioritize defect corrections and enhancement requests. ■ Dynamically adjust the scope of future releases or iterations. ■ Evaluate the impact of proposed changes on users and business processes. ■ Participate in making change decisions.

Multiple product champions

One person can rarely describe the needs for all users of an application. The Chemical Tracking System had four major user classes, so it needed four product champions selected from the internal user community at Contoso Pharmaceuticals. Figure 6-4 illustrates how the project manager set up a team of BAs and product champions to elicit the right requirements from the right sources. These champions were not assigned full time, but each one spent several hours per week working on the project. Three BAs worked with the four product champions to elicit, analyze, and document their requirements. (One BA worked with two product champions because the Buyer and the Health and Safety Department user classes were small and had few requirements.) One of the BAs assembled all the input into a unified SRS.

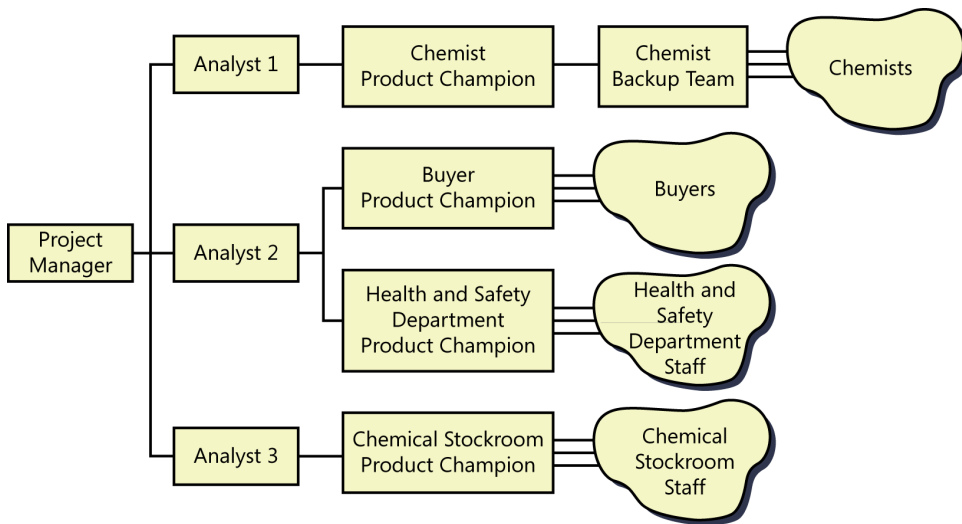


FIGURE 6-4 Product champion model for the Chemical Tracking System.

We didn't expect a single person to provide all the diverse requirements for the hundreds of chemists at Contoso. Don, the product champion for the Chemist user class, assembled a backup

team of five chemists from other parts of the company. They represented subclasses within the broad Chemist user class. This hierarchical approach engaged additional users in requirements development while avoiding the expense of massive workshops or dozens of individual interviews. Don always strove for consensus. However, he willingly made the necessary decisions when agreement wasn't achieved so the project could move ahead. No backup team was necessary when the user class was small enough or cohesive enough that one individual truly could represent the group's needs.¹



The voiceless user class

A business analyst at Humongous Insurance was delighted that an influential user, Rebecca, agreed to serve as product champion for the new claims processing system. Rebecca had many ideas about the system features and user interface design. Thrilled to have the guidance of an expert, the development team happily complied with her requests. After delivery, though, they were shocked to receive many complaints about how hard the system was to use.

Rebecca was a power user. She specified usability requirements that were great for experts, but the 90 percent of users who *weren't* experts found the system unintuitive and difficult to learn. The BA didn't recognize that the claims processing system had at least two user classes. The large group of non-power users was disenfranchised in the requirements and user interface design processes. Humongous paid the price in an expensive redesign. The BA should have engaged at least one more product champion to represent the large class of nonexpert users.

Selling the product champion idea

Expect to encounter resistance when you propose the idea of having product champions on your projects. "The users are too busy." "Management wants to make the decisions." "They'll slow us down." "We can't afford it." "They'll run amok and scope will explode." "I don't know what I'm supposed to do as a product champion." Some users won't want to cooperate on a project that will make them change how they work or might even threaten their jobs. Managers are sometimes reluctant to delegate authority for requirements to ordinary users.

Separating business requirements from user requirements alleviates some of these discomforts. As an actual user, the product champion makes decisions at the user requirements level within the scope boundaries imposed by the business requirements. The management sponsor retains the authority to make decisions that affect the product vision, project scope, business-related priorities, schedule, or budget. Documenting and negotiating each product champion's role and responsibilities give candidate champions a comfort level about what they're being asked to do. Remind management that a product champion is a key contributor who can help the project achieve its business objectives.

¹ There's an interesting coda to this story. Years after I worked on this project, a man in a class I was teaching said he had worked at the company that Contoso Pharmaceuticals had contracted to build the Chemical Tracking System. The developers found that the requirements specification we created using this product champion model provided a solid foundation for the development work. The system was delivered successfully and was used at Contoso for many years.

If you encounter resistance, point out that insufficient user involvement is a leading cause of software project failure. Remind the protesters of problems they've experienced on previous projects that trace back to inadequate user input. Every organization has horror stories of new systems that didn't satisfy user needs or failed to meet unstated usability or performance expectations. You can't afford to rebuild or discard systems that don't measure up because no one understood the requirements. Product champions provide one way to get that all-important customer input in a timely way, not at the end of the project when customers are disappointed and developers are tired.

Product champion traps to avoid

The product champion model has succeeded in many environments. It works only when the product champions understand and sign up for their responsibilities, have the authority to make decisions at the user requirements level, and have time available to do the job. Watch out for the following potential problems:

- Managers override the decisions that a qualified and duly authorized product champion makes. Perhaps a manager has a wild new idea at the last minute, or thinks he knows what the users need. This behavior often results in dissatisfied users and frustrated product champions who feel that management doesn't trust them.
- A product champion who forgets that he is representing other customers and presents only his own requirements won't do a good job. He might be happy with the outcome, but others likely won't be.
- A product champion who lacks a clear vision of the new system might defer decisions to the BA. If all of the BA's ideas are fine with the champion, the champion isn't providing much help.
- A senior user might nominate a less experienced user as champion because she doesn't have time to do the job herself. This can lead to backseat driving from the senior user who still wishes to strongly influence the project's direction.

Beware of users who purport to speak for a user class to which they do not belong. Rarely, an individual might actively try to block the BA from working with the ideal contacts for some reason. On the Chemical Tracking System, the product champion for the chemical stockroom staff—herself a former chemist—initially insisted on providing what she thought were the needs of the chemist user class. Unfortunately, her input about current chemist needs wasn't accurate. It was difficult to convince her that this wasn't her job, but the BA didn't let her intimidate him. The project manager lined up a separate product champion for the chemists, who did a great job of collecting, evaluating, and relaying that community's requirements.



User representation on agile projects

Frequent conversations between project team members and appropriate customers are the most effective way to resolve many requirements issues and to flesh out requirements specifics when they are needed. Written documentation, however detailed, is an incomplete substitute for these ongoing communications. A fundamental tenet of Extreme Programming, one of the early agile development methods, is the presence of a full-time, on-site customer for these discussions (Jeffries, Anderson, and Hendrickson, 2001).

Some agile development methods include a single representative of stakeholders called a *product owner* in the team to serve as the voice of the customer (Schwaber 2004; Cohn 2010; Leffingwell 2011). The product owner defines the product's vision and is responsible for developing and prioritizing the contents of the product backlog. (The *backlog* is the prioritized list of user stories—requirements—for the product and their allocation to upcoming iterations, called sprints in the agile development method called Scrum.) The product owner therefore spans all three levels of requirements: business, user, and functional. He essentially straddles the product champion and business analyst functions, representing the customer, defining product features, prioritizing them, and so forth. Ultimately, someone does have to make decisions about exactly what capabilities to deliver in the product and when. In Scrum, that's the product owner's responsibility.



The ideal state of having a single product owner isn't always practical. We know of one company that was implementing a package solution to run their insurance business. The organization was too big and complex to have one person who understood everything in enough detail to make all decisions about the implementation. Instead, the customers selected a product owner from each department to own the priorities for the functionality used by that department. The company's CIO served as the lead product owner. The CIO understood the entire product vision, so he could ensure that the departments were on track to deliver that vision. He had responsibility for decision making when there were conflicts between department-level product owners.

The premises of the on-site customer and close customer collaboration with developers that agile methods espouse certainly are sound. In fact, we feel strongly that *all* development projects warrant this emphasis on user involvement. As you have seen, though, all but the smallest projects have multiple user classes, as well as numerous additional stakeholders whose interests must be represented. In many cases it's not realistic to expect a single individual to be able to adequately understand and describe the needs of all relevant user classes, nor to make all the decisions associated with product definition. Particularly with internal corporate projects, it will generally work better to use a representative structure like the product champion model to ensure adequate user engagement.

The product owner and product champion schemes are not mutually exclusive. If the product owner is functioning in the role of a business analyst, rather than as a stakeholder representative himself, he could set up a structure with one or more product champions to see that the most appropriate sources provide input. Alternatively, the product owner could collaborate with one or more business analysts, who then work with stakeholders to understand their requirements. The product owner would then serve as the ultimate decision maker.



“On-sight” customer

I once wrote programs for a research scientist who sat about 10 feet from my desk. John could provide instantaneous answers to my questions, provide feedback on user interface designs, and clarify our informally written requirements. One day John moved to a new office, around the corner on the same floor of the same building, about 100 feet away. I perceived an immediate drop in my programming productivity because of the cycle time delay in getting John’s input. I spent more time fixing problems because sometimes I went down the wrong path before I could get a course correction. There’s no substitute for having the right customers continuously available to the developers both on-site and “on-sight.” Beware, though, of too-frequent interruptions that make it hard for people to refocus their attention on their work. It can take up to 15 minutes to reimmerge yourself into the highly productive, focused state of mind called *flow* (DeMarco and Lister 1999).



An on-site customer doesn’t guarantee the desired outcome. My colleague Chris, a project manager, established a development team environment with minimal physical barriers and engaged two product champions. Chris offered this report: “While the close proximity seems to work for the development team, the results with product champions have been mixed. One sat in our midst and still managed to avoid us all. The new champion does a fine job of interacting with the developers and has truly enabled the rapid development of software.” There is no substitute for having the right people, in the right role, in the right place, with the right attitude.

Resolving conflicting requirements

Someone must resolve conflicting requirements from different user classes, reconcile inconsistencies, and arbitrate questions of scope that arise. The product champions or product owner can handle this in many, but likely not all, cases. Early in the project, determine who the decision makers will be for requirements issues, as discussed in Chapter 2. If it’s not clear who is responsible for making these decisions or if the authorized individuals abdicate their responsibilities, the decisions will fall to the developers or analysts by default. Most of them don’t have the necessary knowledge and perspective

to make the best business decisions, though. Analysts sometimes defer to the loudest voice they hear or to the person highest on the food chain. Though understandable, this is not the best strategy. Decisions should be made as low in the organization's hierarchy as possible by well-informed people who are close to the issues.

Table 6-3 identifies some requirements conflicts that can arise on projects and suggests ways to handle them. The project's leaders need to determine who will decide what to do when such situations arise, who will make the call if agreement is not reached, and to whom significant issues must be escalated when necessary.

TABLE 6-3 Suggestions for resolving requirements disputes

Disagreement between	How to resolve
Individual users	Product champion or product owner decides
User classes	Favored user class gets preference
Market segments	Segment with greatest impact on business success gets preference
Corporate customers	Business objectives dictate direction
Users and user managers	Product owner or product champion for the user class decides
Development and customers	Customers get preference, but in alignment with business objectives
Development and marketing	Marketing gets preference

Trap Don't justify doing whatever any customer demands because "The customer is always right." We all know the customer is *not* always right (Wiegers 2011). Sometimes, a customer is unreasonable, uninformed, or in a bad mood. The customer always has a point, though, and the software team must understand and respect that point.

These negotiations don't always turn out the way the analyst might hope. Certain customers might reject all attempts to consider reasonable alternatives and other points of view. We've seen cases where marketing never said no to a customer request, no matter how infeasible or expensive. The team needs to decide who will be making decisions on the project's requirements before they confront these types of issues. Otherwise, indecision and the revisiting of previous decisions can stall the project in endless wrangling. If you're a BA caught in this dilemma, rely on your organizational structure and processes to work through the disagreements. But, as we've cautioned before, there aren't any easy solutions if you're working with truly unreasonable people.



Next steps

- Relate Figure 6-3 to the way you hear the voice of the user in your own environment. Do you encounter any problems with your current communication links? Identify the shortest and most effective communication paths that you can use to elicit user requirements in the future.
- Identify the different user classes for your project. Which ones are favored? Which, if any, are disfavored? Who would make a good product champion for each important user class? Even if the project is already underway, the team likely would benefit from having product champions involved.
- Starting with Table 6-2, define the activities you would like your product champions to perform. Negotiate the specific contributions with each candidate product champion and his or her manager.
- Determine who the decision makers are for requirements issues on your project. How well does your current decision-making approach work? Where does it break down? Are the right people making decisions? If not, who should be doing it? Suggest processes that the decision makers should use for reaching agreement on requirements issues.